Temporal-Difference Learning (TD)

First, a story

You walk the same route to school every day. Today, halfway there, you find the road closed; the detour costs ten extra minutes. How should you update your prediction of arrival time?

- **Method A**: complete the whole route, look at the clock at home, update your prediction with the actual time. Accurate — but you must wait until arrival;
- **Method B**: halfway through, glance at the remaining road, estimate today's total, and revise on the spot. No waiting — but the guess may be biased.

Method A is Monte Carlo (MC); method B is **temporal-difference learning** (TD). TD's core idea: **don't wait for the outcome** — after every step, correct the current estimate using "immediate feedback plus a guess about the future". You do not need the full truth; one step of feedback and your current prediction suffice.

Formalization: the TD(0) update

Recall the two extremes:

Monte Carlo — run the whole trajectory, update with the true return:

$$V(S_t) \leftarrow V(S_t) + \alpha\big[G_t - V(S_t)\big], \qquad G_t = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{T-t-1} R_T$$

G_t is an **unbiased** estimate of $v_\pi(S_t)$ but has high variance (a whole trajectory fluctuates), and you must wait for the episode to end.

Dynamic programming — with a known model, take the expectation over all successors:

$$V(s) \leftarrow \sum_a \pi(a \mid s)\Big[R(s,a) + \gamma \sum_{s'} P(s' \mid s,a)\, V(s')\Big]$$

Zero variance, deterministic — but requires the full model $P(s' \mid s, a)$, rarely available in reality.

TD(0) takes the best of both — each step uses **one real reward** and **the current estimate of the successor**:

$$\boxed{V(S_t) \leftarrow V(S_t) + \alpha\Big[\underbrace{R_{t+1} + \gamma V(S_{t+1})}_{\text{TD target}} - V(S_t)\Big]}$$

- $R_{t+1} + \gamma V(S_{t+1})$ — the **TD target**: one real reward plus the current estimate of the successor, approximating G_t;
- $R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$ — the **TD error**: the gap between the current estimate and a "fresher" one;
- vs. MC: MC uses the true G_t; TD replaces the unknown post-$t{+}1$ rewards with the estimate $V(S_{t+1})$ — that is **bootstrapping** (updating an estimate from an estimate);
- vs. DP: DP's expectation over successors needs $P(s' \mid s, a)$; TD **samples a single actual** S_{t+1} — no model required.

The TD error and the source of "learning fast"

A rigorous definition

At time t the agent is in S_t; after acting it receives R_{t+1} and moves to S_{t+1}. The TD error is:

$$\boxed{\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)}$$

If $V = v_\pi$ (the true value function), then $\mathbb{E}[\delta_t \mid S_t = s] = 0$ — an immediate corollary of the Bellman equation: $v_\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s]$. As V approaches v_π, δ_t tends to zero mean. TD learning is the process of driving δ_t toward zero.

MC vs TD(0): the bias–variance trade-off

	MC	TD(0)
Update target	G_t (true return)	$R_{t+1} + \gamma V(S_{t+1})$ (one-step estimate)
Bootstrapping	no	yes
Bias	unbiased	biased
Variance	high	low
Must wait for episode end	yes	no
Needs a model	no	no

TD accepts a **biased** estimate in exchange for **much lower variance** and faster convergence. On the 5-state random walk (Figure 1), both methods consume the same batch of trajectories: MC must average returns with a small step size ($\alpha = 0.01$) and lags; TD(0) corrects in place every step with a larger step size ($\alpha = 0.1$) and tracks the true values sooner.

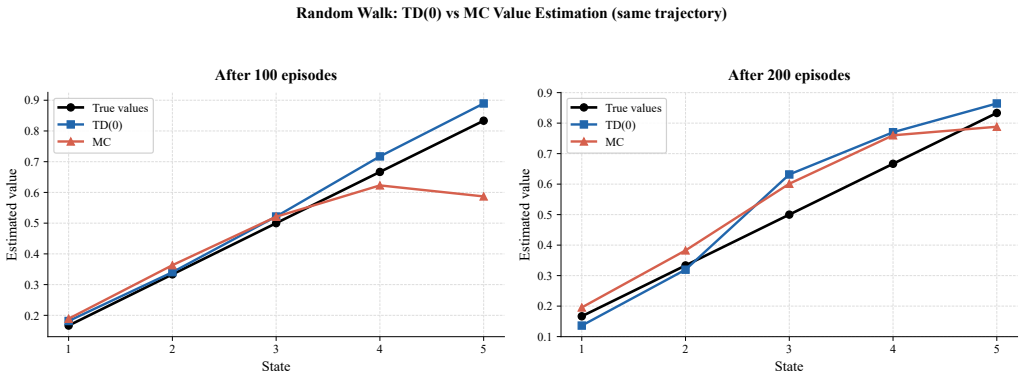

Figure 1: Value estimates on the 5-state random walk (same batch of trajectories). The black line is the true value $v(s) = s/6$; red is MC ($\alpha = 0.01$), blue is TD(0) ($\alpha = 0.1$). TD(0) is closer to the truth after both 100 and 200 episodes (100-episode average RMS: MC 0.149 vs TD 0.062).

From Prediction to Control: SARSA and Q-Learning

The previous section used V for prediction; this section uses Q for control (learning a policy).

On-policy: SARSA

SARSA learns the q_π of the policy **actually executed** (including exploration):

$$\boxed{Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\big[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\big]}$$

Actions are usually chosen by ε-greedy. The SARSA loop:

1. in S_t, pick A_t ε-greedily;
2. execute A_t, receive R_{t+1}, arrive at S_{t+1};
3. in S_{t+1}, **pick again** A_{t+1} ε-greedily (not executed; only its Q value is used);
4. update $Q(S_t, A_t)$ with A_{t+1}'s Q value;
5. $S_t \leftarrow S_{t+1}$, $A_t \leftarrow A_{t+1}$, repeat.

The key: A_{t+1} in the target comes from the **actually executed policy** (with exploration), so SARSA learns the q_π of the ε-greedy policy, **accounting for the cost of exploration**.

Off-policy: Q-Learning

Q-Learning learns the **optimal** (greedy) policy while still exploring during execution:

$$\boxed{Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\big[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)\big]}$$

The only difference from SARSA: the target uses $\max_a Q(S_{t+1}, a)$ instead of $Q(S_{t+1}, A_{t+1})$. SARSA's target depends on the actually sampled A_{t+1} (exploration noise included); Q-Learning's target **always assumes the best action next**, whatever exploration actually did. Hence off-policy: the **policy learned** (greedy, optimal) differs from the **policy executed** (ε-greedy, exploring).

SARSA vs Q-Learning: Cliff Walking

The classic Cliff Walking environment (4×12 grid, start bottom-left, goal bottom-right; the 10 middle cells of the bottom row are cliff: stepping on one returns to the start with -100; ordinary steps cost -1) exposes the difference (Figure 2).

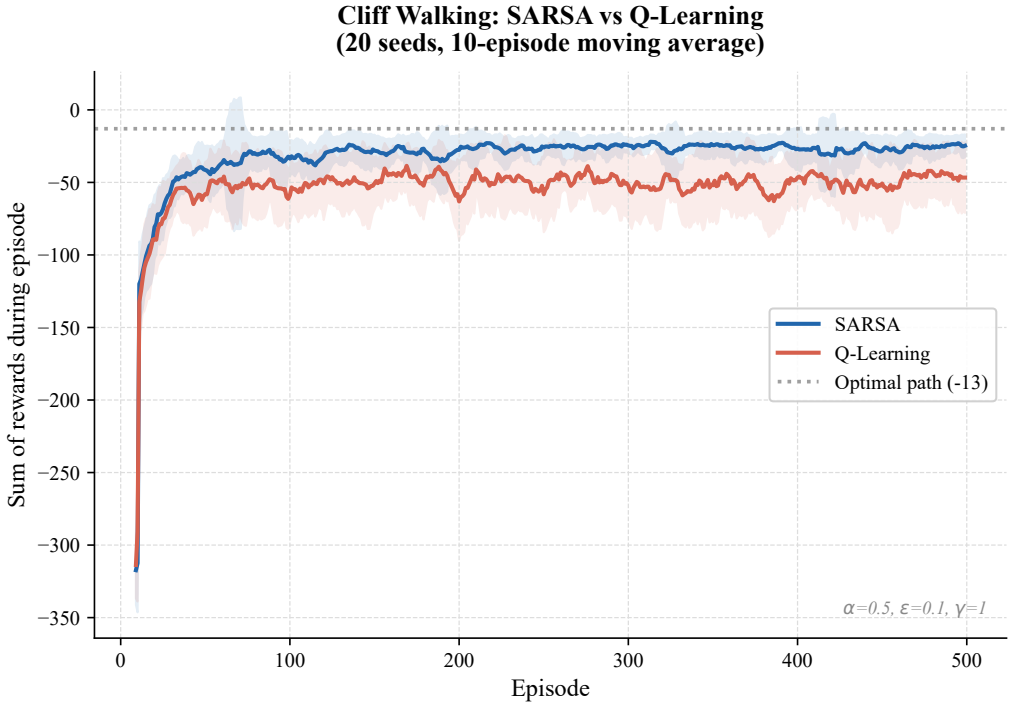

Figure 2: Cliff Walking (4×12, $\alpha = 0.5$, $\varepsilon = 0.1$, $\gamma = 1$, 20 random seeds, 10-episode moving average). During training SARSA's per-episode reward is markedly higher: SARSA learns a conservative detour (occasional exploration is not catastrophic), Q-Learning learns the shortest path but repeatedly falls off the cliff during training due to random exploration.

The key insight: Q-Learning assumes "from now on I always act optimally", so hugging the cliff seems fine (the optimal policy never steps off); SARSA knows that in deployment it will occasionally explore, and one random move along the cliff is a disaster — hence a safer, more conservative policy. If at deployment the agent is fully greedy ($\varepsilon = 0$), Q-Learning's aggressive policy may be better — it depends on whether your agent can still make mistakes after deployment.

Expected SARSA and the Softmax policy

Expected SARSA avoids sampling A_{t+1} by taking the expectation over all actions, removing sampling variance:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma \sum_a \pi(a \mid S_{t+1})\, Q(S_{t+1}, a) - Q(S_t, A_t)\Big]$$

With an ε-greedy target policy it is on-policy; with a purely greedy π it **reduces to Q-Learning** (off-policy). On most tasks Expected SARSA beats plain SARSA — a tiny compute overhead is repaid many times over by variance reduction.

The Softmax policy distributes exploration probability by relative Q values with temperature τ:

$$\pi(a \mid s) = \frac{\exp\big(Q(s,a)/\tau\big)}{\sum_b \exp\big(Q(s,b)/\tau\big)}$$

Large τ approaches uniform exploration; $\tau \to 0$ approaches pure greedy. Exploration is no longer blind (ε-greedy picks every non-greedy action with equal probability $\varepsilon/|\mathcal{A}|$, however obviously bad): **better actions get more probability**.

Multi-step Extensions: n-step TD and TD(λ)

The n-step return

TD(0) looks one step ahead (low variance, high bias); MC looks the whole way (unbiased, high variance). **n-step TD** spans the spectrum; define the n-step return:

$$\boxed{G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})}$$

- $n = 1$: $G_t^{(1)} = R_{t+1} + \gamma V(S_{t+1})$ — TD(0);
- $n \to \infty$: $G_t^{(\infty)} = G_t$ — MC;
- $1 < n < \infty$: n real rewards, then V estimates the remainder at the truncation point.

Update: $V(S_t) \leftarrow V(S_t) + \alpha\big[G_t^{(n)} - V(S_t)\big]$. n-step SARSA is analogous with Q in place of V; replacing the last term by $\sum_a \pi(a \mid S_{t+n}) Q(S_{t+n}, a)$ gives Expected n-step SARSA.

Figure 3 shows how n moves the bias–variance dial: smaller n (more bootstrapping) yields lower error here; larger n (more real rewards) approaches MC's higher error.

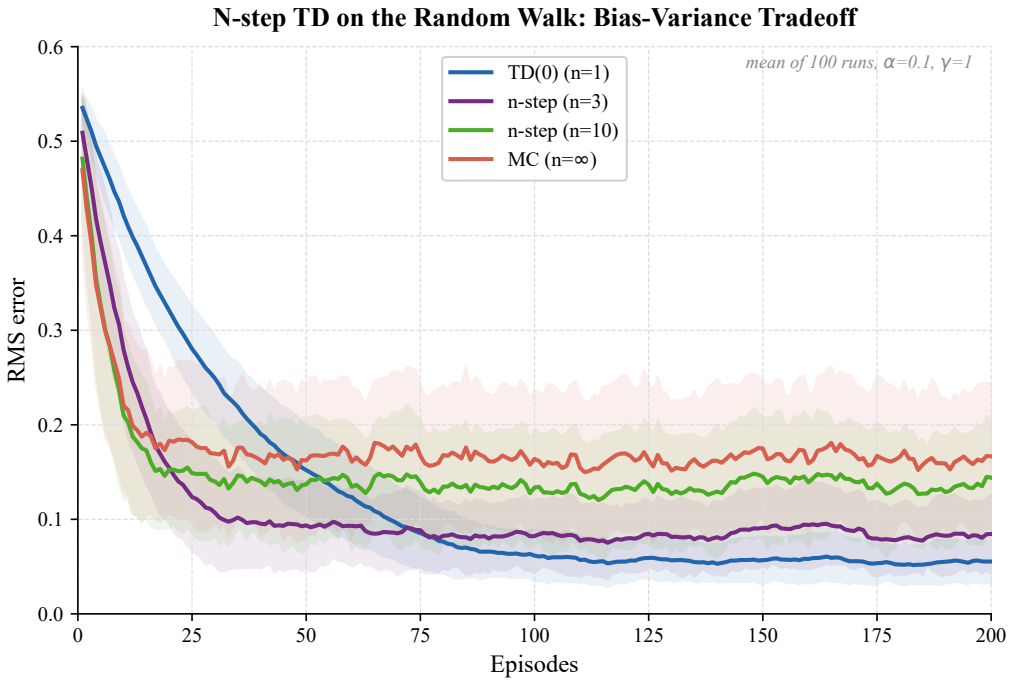

Figure 3: RMS error of n-step TD on the 5-state random walk vs. number of episodes (same $\alpha = 0.1$, averaged over 100 episodes). $n = 1$ (TD(0)) is lowest; $n = 3$ and $n = 10$ rise; $n \to \infty$ (MC) is highest — smaller n bootstraps more, lowers variance, learns faster.

TD(λ)

Rather than a single n, TD(λ) mixes all n-step returns with **geometric weights**:

$$\boxed{G_t^\lambda = (1-\lambda)\sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}}$$

- $\lambda = 0$: $G_t^\lambda = G_t^{(1)}$ — TD(0);
- $\lambda = 1$: $G_t^\lambda = G_t$ — MC;
- $0 < \lambda < 1$: nearer steps get more weight; farther steps contribute less but not zero.

λ controls the degree of bootstrapping: smaller leans on estimates (high bias, low variance), larger leans on real rewards (low bias, high variance).

Eligibility traces: the online implementation of TD(λ)

Computing G_t^λ requires the whole episode. **Eligibility traces** give an online incremental form: visiting a state–action pair (s, a) adds 1 to its trace; every step all traces decay by $\gamma\lambda$:

$$e_t(s,a) = \begin{cases} \gamma\lambda\, e_{t-1}(s,a) + 1, & \text{if } (s,a) \text{ is the current pair,} \\ \gamma\lambda\, e_{t-1}(s,a), & \text{otherwise.} \end{cases}$$

Update: every step, for all (s, a), $Q(s,a) \leftarrow Q(s,a) + \alpha\, \delta_t\, e(s,a)$. Intuition: the trace is a decaying record of "how responsible this pair is for the current outcome" — recent pairs weigh more, old ones fade to zero.

Watkins' Q(λ): eligibility traces are naturally on-policy while Q-Learning is off-policy, so special handling is required — once a non-greedy (exploratory) action is taken, the future trajectory no longer follows the target policy and the traces must be wiped:

$$\text{if } A_{t+1} \neq \arg\max_a Q(S_{t+1}, a) \implies \text{all } e(s,a) \leftarrow 0$$

The cost: with a high exploration rate, the effective trace length is set by "the run of consecutive greedy actions", not by a fixed λ.

Experience Replay — Data Reuse for Off-Policy

Online learning uses each transition $(S_t, A_t, R_{t+1}, S_{t+1})$ once and discards it — extremely wasteful. **Experience replay** maintains a fixed-size buffer of the most recent N transitions:

1. the acting policy produces new transitions, stored in the buffer;
2. a mini-batch is **sampled uniformly** from the buffer;
3. multiple Q-Learning updates are performed on the sampled transitions.

Why it works:

- **Breaks correlations**: consecutive transitions are highly correlated; uniform sampling shuffles them, reducing update variance;
- **Sample reuse**: each transition is used multiple times, improving data efficiency;
- **Stabilizes training**: replaying "old data" acts as a regularizer.

Limitation: experience replay only suits off-policy algorithms (Q-Learning, Expected SARSA). SARSA cannot use it — its update depends on the current policy's action selection, while buffered data comes from old policies.

Where this chapter sits in the RL landscape

1. **Upstream**: MAB's incremental updates and exploration–exploitation; MDP's Bellman equations and exact DP. DP needs a known model; MC uses true returns (unbiased, high variance, waits for episode end); TD takes the best of both — one-step sampling plus bootstrapping;
2. **This chapter (TD learning)**: model-free value learning at its core — the TD(0) update, the on/off-policy split of SARSA/Q-Learning, n-step and traces tuning the bias–variance dial, and experience replay for off-policy efficiency;
3. **Downstream**: Q-Learning's max operator plus neural-network function approximation equals DQN; replay plus target networks tames the instability of TD updates (next chapter).

The TD algorithm family

Algorithm	on/off-policy	Bootstrap	Target form	Character
MC	on	no	G_t	unbiased, high variance
TD(0)	on	yes	$R + \gamma V(s')$	biased, low variance
SARSA	on	yes	$R + \gamma Q(s', a')$	prices in exploration
Q-Learning	off	yes	$R + \gamma \max_a Q(s', \cdot)$	learns the optimum
Expected SARSA	both	yes	$R + \gamma \sum_a \pi(a \mid s') Q(s', a)$	lower variance
N-step	both	yes	n-step mixed return	tunable bias–variance
Q(λ)	off	yes	λ-weighted return + traces	online multi-step

- **TD(0)**: one real reward plus the next estimate; no waiting for episode end;
- **SARSA**: learn what you execute — the true q_π under ε-greedy, conservative and safe;
- **Q-Learning**: learn q_* directly regardless of exploration;
- **Expected SARSA**: average over the sampled action to cut variance; degenerates to Q-Learning under a greedy π;
- **N-step TD**: trade bias against variance; larger n approaches MC;
- **TD(λ) + traces**: distribute credit over steps with exponential decay; online multi-step propagation;
- **Experience replay**: store old data, replay uniformly — decorrelates and improves sample efficiency (off-policy only).

The bridge from TD to DQN

1. **Q-Learning's** max **operator + function approximation** = DQN;
2. **Experience replay + target networks** = taming the instability of TD updates in DQN;
3. ε-**greedy exploration** carries into DQN (simple but effective; later improved by NoisyNet and others);
4. **Eligibility traces** → Retrace-style algorithms → efficient off-policy multi-step learning with replay.

With this chapter you hold the core of model-free value learning — TD trades bias for variance via bootstrapping; SARSA/Q-Learning split on/off-policy; n-step and traces tune the dial. The next chapter, DQN, replaces the Q table with a neural network and tames it with replay and target networks.

References

1. Sutton & Barto (2018). *Reinforcement Learning: An Introduction* (2nd ed.). Chapters 6–7, 12.
2. Watkins, C. J. C. H. (1989). *Learning from Delayed Rewards*. PhD Thesis. (The original Q-Learning paper.)

3. Rummery & Niranjan (1994). On-line Q-learning using connectionist systems. (The original SARSA paper.)

4. van Seijen et al. (2009). A theoretical and empirical analysis of Expected Sarsa.

5. Lin (1992). Self-improving reactive agents based on reinforcement learning. (Invention of experience replay.)

6. Mnih et al. (2015). Human-level control through deep reinforcement learning. *Nature*. (DQN + experience replay.)

7. Practical RL, Yandex Data School. Week 3 — Model-Free Reinforcement Learning.